COMPREHENSIVE GUIDE

System Architecture

Building Scalable, Reliable & High-Performance Systems

18 Chapters
From basics to advanced

6 Case Studies
Netflix, YouTube & more

50+ Diagrams
Visual architecture maps

April 2026 Edition

Table of Contents

PART I — FUNDAMENTALS

01 Introduction to System Architecture

02 Fundamental Building Blocks

03 Core Architecture Patterns

04 Scaling Strategies

PART II — CORE INFRASTRUCTURE

05 Databases & Storage

06 Caching Strategies

07 Message Queues & Event Streaming

08 Content Delivery Networks (CDNs)

09 Microservices Architecture

PART III — REAL-WORLD CASE STUDIES

10 **Case Study: Netflix**

11 **Case Study: YouTube**

12 **Case Study: Twitter / X**

13 **Case Study: Amazon**

14 **Case Study: Uber**

15 **Case Study: WhatsApp**

PART IV — BEST PRACTICES

16 System Design Best Practices

17 Monitoring, Observability & Reliability

18 Summary & Key Takeaways

1. Introduction to System Architecture

What is System Architecture?

System architecture is the **conceptual model** that defines the structure, behavior, and relationships of a system's components. It encompasses how software components, hardware infrastructure, and network layers interact to deliver a service.

Think of it as the **blueprint of a building** — before you lay a single brick, you need to know where the walls go, how the plumbing works, and how many floors you need.

Why Does It Matter?

Concern	Impact of Poor Architecture
Scalability	System crashes under high traffic
Reliability	Frequent outages and data loss
Performance	Slow response times, poor user experience
Maintainability	Bugs are hard to fix, features take months to ship
Cost	Over-provisioned or under-utilized resources

The Scale Spectrum

HOW ARCHITECTURE EVOLVES WITH USER COUNT

```
Small Scale            Medium Scale            Large Scale
(100s users)           (100K users)            (100M+ users)
    |                      |                       |
 Single Server          Load Balanced            Distributed
 Monolith App           Multiple Servers          Microservices
 Single DB              Read Replicas             Sharded DBs
 No Cache               Redis Cache               Multi-layer Cache
 No CDN                 Basic CDN                 Global CDN
```

Key Insight: *Architecture should evolve with your scale. Don't over-engineer early, but design with growth in mind.*

2. Fundamental Building Blocks

2.1 Client-Server Model

The foundation of web architecture. Every interaction starts here.

REQUEST-RESPONSE CYCLE

```
┌──────────┐                 HTTP Request          ┌──────────┐
│  Client  │ ──────────────────────────────────►   │  Server  │
│ (Browser)│                                       │  (App)   │
│          │ ◄──────────────────────────────────   │          │
└──────────┘                 HTTP Response         └──────────┘
```

1. Client sends an HTTP request (GET, POST, PUT, DELETE)
2. Server processes the request (business logic, DB queries)
3. Server returns an HTTP response (HTML, JSON, etc.)

2.2 DNS (Domain Name System)

DNS translates human-readable domain names into IP addresses.

DNS RESOLUTION FLOW

```
User types "netflix.com"
        │
        ▼
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ DNS Resolver │──►│Root Server   │──►│ TLD Server   │
│ (ISP/Google) │   │    (.)       │   │   (.com)     │
└──────────────┘   └──────────────┘   └──────┬───────┘
        │                                     │
        │                                     │
        └──────────► ┌──────────────┐         │
                     │Authoritative NS│◄──────┘
                     │(netflix.com)   │
                     │→ 54.85.xxx.xxx │
                     └──────────────┘
```

- **Round-Robin DNS:** Distributes traffic across multiple IPs
- **GeoDNS:** Routes to the nearest data center based on user location
- **Weighted DNS:** Sends a percentage of traffic to different servers

2.3 Common Ports

Port	Service	Port	Service
80	HTTP	443	HTTPS
3306	MySQL	5432	PostgreSQL
6379	Redis	27017	MongoDB

2.4 APIs (Application Programming Interfaces)

REST API

```
GET    /api/users       → List users
GET    /api/users/123   → Get user
POST   /api/users       → Create user
PUT    /api/users/123   → Update user
DELETE /api/users/123   → Delete user
```

GraphQL

```
query {
  user(id: "123") {
    name
    email
    posts { title, likes }
  }
}
```

gRPC (Google Remote Procedure Call)

- Uses **Protocol Buffers** (binary serialization) — up to **10x faster** than REST
- Supports **bidirectional streaming**
- Used heavily by Google, Netflix, and Uber internally

2.5 Protocols Comparison

Protocol	Use Case	Key Feature
HTTP/1.1	Standard web	Text-based, one request per connection
HTTP/2	Modern web	Multiplexing, header compression
HTTP/3	Next-gen web	UDP-based (QUIC), faster connections
WebSocket	Real-time apps	Full-duplex, persistent connection
TCP	Reliable transfer	Guaranteed delivery, ordered packets
UDP	Streaming / gaming	Fast, no guarantee, unordered

3. Core Architecture Patterns

3.1 Monolithic Architecture

Everything runs as a single application.

```
┌──────────────────────────────────────┐
│            MONOLITH APP              │
│                                      │
│  ┌─────────┐  ┌─────────┐  ┌────────┐  │
│  │  User   │  │ Order   │  │Payment │  │
│  │ Module  │  │ Module  │  │ Module │  │
│  └─────────┘  └─────────┘  └────────┘  │
│                                      │
│  ┌─────────┐  ┌─────────┐  ┌────────┐  │
│  │  Auth   │  │ Search  │  │ Email  │  │
│  │ Module  │  │ Module  │  │ Module │  │
│  └─────────┘  └─────────┘  └────────┘  │
│       Shared Database Connection     │
└──────────────────────────────────────┘
                  │
              ┌─────────┐
              │ Database │
              └─────────┘
```

Pros

- Simple to develop and test
- Easy to deploy initially
- Simple debugging

Cons

- Becomes "big ball of mud" at scale
- One module failure crashes everything
- Hard to scale individual components

3.2 Microservices Architecture

Each service is independently deployable, scalable, and maintainable.

MICROSERVICES WITH INDEPENDENT DATABASES

```
              ┌──────────┐
              │API Gateway│
              └──────────┘
                   │
        ┌──────────┼──────────┐
        ▼          ▼          ▼
   ┌────────┐  ┌────────┐  ┌────────┐
   │User Svc│  │Order   │  │Payment │
   │        │  │  Svc   │  │  Svc   │
   └────────┘  └────────┘  └────────┘
       │           │           │
   ┌───┴───┐   ┌───┴───┐   ┌───┴───┐
   │UserDB│   │OrdDB │   │PayDB │
   └───────┘   └───────┘   └───────┘
```

- **Single Responsibility:** Each service does one thing well
- **Decentralized Data:** Each service owns its database
- **Independent Deployment:** Deploy without affecting others
- **Failure Isolation:** One service failure doesn't cascade

3.3 Event-Driven Architecture

ASYNCHRONOUS EVENT FLOW

```
┌──────────┐  Event  ┌──────────────┐  Event  ┌──────────┐
│ Producer │────────►│  Event Bus   │────────►│Consumer A│
│ Service  │         │/ Msg Queue   │         └──────────┘
└──────────┘         └──────────────┘
                            │
                            │ Event
                            ▼
                     ┌──────────────┐
                     │Consumer B    │
                     └──────────────┘
```

Example — User places an order:

1. **OrderService** publishes `OrderCreated` event
2. **PaymentService** consumes it → processes payment
3. **InventoryService** consumes it → reduces stock
4. **NotificationService** consumes it → sends confirmation email

4. Scaling Strategies

4.1 Vertical vs Horizontal Scaling

Vertical (Scale Up)

```
Before:          After:
┌──────────┐     ┌──────────────┐
│ 2 cores  │     │  32 cores    │
│ 4 GB RAM │ ──► │  128 GB RAM  │
│ 100 GB   │     │  2 TB NVMe   │
└──────────┘     └──────────────┘
```

- Simple, no code changes
- Hardware limits exist
- Single point of failure

Horizontal (Scale Out)

```
          ┌──────────────┐
          │Load Balancer │
          └──────────────┘
             │     │     │
             ▼     ▼     ▼
       ┌─────┐ ┌─────┐ ┌─────┐
       │Srv 1│ │Srv 2│ │Srv 3│
       └─────┘ └─────┘ └─────┘
```

- Virtually unlimited scale
- Built-in redundancy
- More complex to manage

4.2 Load Balancing Algorithms

Algorithm	Description	Best For
Round Robin	Rotate through servers sequentially	Equal-capacity servers
Weighted Round Robin	Rotate with weight preferences	Mixed-capacity servers
Least Connections	Route to fewest active connections	Varying request complexity
IP Hash	Hash client IP to pick server	Session persistence

Layer 4 vs Layer 7 Load Balancing

Layer 4 (Transport)

- Routes based on IP + Port
- Faster, less intelligent
- Example: AWS NLB

Layer 7 (Application)

- Routes by HTTP headers, URL, cookies
- Slower, much more flexible
- Example: AWS ALB, Nginx

```
Layer 7 Routing Example:
  /api/users   →  User Service Cluster
  /api/orders  →  Order Service Cluster
  /static/*    →  CDN / Static Server
```

4.3 Auto-Scaling

```
Traffic Load:
  ▓▓▓
```

Relational Databases (SQL)

```
┌──────────────────────────────────────────┐
│  users table                             │
├──────┬──────────┬──────────────┬──────────┤
│  id  │  name    │  email       │ created_at│
├──────┼──────────┼──────────────┼──────────┤
│  1   │  Alice   │  a@mail.com  │ 2024-01-01│
│  2   │  Bob     │  b@mail.com  │ 2024-01-02│
└──────┴──────────┴──────────────┴──────────┘
         │  JOIN on user_id
         ▼
┌──────────────────────────────────────────┐
│  orders table                            │
├──────┬──────────┬──────────┬──────────────┤
│  id  │  user_id │  total   │  status      │
├──────┼──────────┼──────────┼──────────────┤
│  1   │  1       │  $99.99  │  completed   │
│  2   │  1       │  $49.99  │  pending     │
└──────┴──────────┴──────────┴──────────────┘
```

Examples: PostgreSQL, MySQL, Oracle | **Best for:** Transactions, complex queries, data integrity

NoSQL Database Types

Document Store (MongoDB)

```
{
  "_id": "user_001",
  "name": "Alice",
  "orders": [
    {"id":"ord_1","total":99.99},
    {"id":"ord_2","total":49.99}
  ]
}
```

Key-Value Store (Redis)

```
Key: "session:abc123"
Val: {userId:1, expires:"..."}

Key: "cache:user:1"
Val: {name:"Alice", ...}
```

Wide-Column (Cassandra)

```
Row Key: "user_1"
  profile → name:"Alice"
            email:"a@mail.com"
  activity→ last_login:"2024-03"
            views: 1543
```

Graph Database (Neo4j)

```
(Alice)--[:FOLLOWS]-->(Bob)
(Alice)--[:LIKES]-->(Post_1)
(Bob)--[:AUTHORED]-->(Post_1)
```

5.2 Database Scaling Patterns

Read Replicas

```
Writes ──────► ┌──────────┐
               │ Primary  │
               │ (Master) │
               └──────┬───┘
           Replication │
          ┌────────────┼────────────┐
          ▼            ▼            ▼
    ┌─────────┐  ┌─────────┐  ┌─────────┐
    │Replica 1│  │Replica 2│  │Replica 3│
    │ (Read)  │  │ (Read)  │  │ (Read)  │
    └─────────┘  └─────────┘  └─────────┘
```

Database Sharding

```
User ID 1-1M       User ID 1M-2M      User ID 2M-3M

┌──────────┐        ┌──────────┐        ┌──────────┐
│ Shard 1  │        │ Shard 2  │        │ Shard 3  │
└──────────┘        └──────────┘        └──────────┘

Strategies:
  Hash-based:   shard = hash(user_id) % num_shards
  Range-based:  Shard 1 = A-H, Shard 2 = I-P, Shard 3 = Q-Z
  Geographic:   shard_us, shard_eu, shard_asia
```

The CAP Theorem

In a distributed system, you can only guarantee **TWO of three**:

Combination	Description	Examples
CP	Consistent + Partition Tolerant	MongoDB, HBase, Redis
AP	Available + Partition Tolerant	Cassandra, DynamoDB, CouchDB
CA	Not practical in distributed systems	Single-node RDBMS only

6. Caching Strategies

6.1 The Caching Hierarchy

FROM FASTEST TO SLOWEST

```
Fastest ──────────────────────────────────────── Slowest

Browser Cache → CDN Cache → App Cache  → DB Cache → Disk
   ~1ms          ~10ms     ~1-5ms(Redis) ~5-10ms   ~1-10ms
  (local)       (edge)     (in-memory)  (buffer)    (I/O)
```

6.2 Caching Patterns

Cache-Aside (Lazy Loading)

```
┌─────┐    Get    ┌───────┐
│ App │──────────►│ Cache │──► HIT? Return immediately.
│     │◄──────────│(Redis)│
│     │           └───────┘
│     │  MISS                ┌─────┐
│     │──────────────────────►│ DB  │
│     │◄──────────────────────│     │
└─────┘  Store in cache       └─────┘
```

Write-Through Cache

```
App ──write──► Cache ──write──► DB
App ◄──read── Cache (always fresh)
```

Consistent but slower writes

Write-Behind Cache

```
App ──write──► Cache ──async──► DB
               (fast)  (batched)
```

Fast writes but risk of data loss

Cache Invalidation Strategies

Strategy	Description	Trade-off
TTL (Time-to-Live)	Data expires after N seconds	Simple but stale data possible
Event-based	Invalidate on write/update events	Fresh but complex to implement
Version-based	Cache key includes version number	Clean but needs versioning system

> *"There are only two hard things in Computer Science: cache invalidation and naming things." — Phil Karlton*

7. Message Queues & Event Streaming

7.1 Message Queues

```
┌──────────┐  Message  ┌──────────┐  Message  ┌──────────┐
│ Producer │──────────►│   Queue  │──────────►│ Consumer │
│ (Order   │           │(RabbitMQ)│           │ (Payment │
│  Service)│           └──────────┘           │  Service)│
└──────────┘                                  └──────────┘
```

- Messages processed **once** then removed
- FIFO ordering (First In, First Out)
- Decouples producers from consumers
- Buffers traffic spikes

Tools: RabbitMQ, Amazon SQS, Azure Service Bus

7.2 Event Streaming (Apache Kafka)

KAFKA: MULTIPLE CONSUMERS, RETAINED EVENTS

```
Producers:          KAFKA CLUSTER              Consumers:

Order Svc  ──►  Topic: "user-events"
                 Partition 0: [E1][E2][E3][E4]  ──► Analytics
User Svc   ──►  Partition 1: [E5][E6][E7]      ──► Search Index
Payment    ──►  Partition 2: [E8][E9]           ──► Notifications
                                                 ──► Fraud Detection
```

- Events are **retained** (replay is possible)
- Multiple consumers can read the **same events**
- Partitions enable **parallel processing**

> ***Kafka at Scale:*** *LinkedIn processes 7 trillion messages/day. Netflix and Uber each process trillions of events through Kafka.*

Queue vs Stream Comparison

Feature	Message Queue	Event Stream
Retention	Deleted after consumption	Retained for configurable time
Consumers	Single consumer per message	Multiple consumers (fan-out)
Replay	Not possible	Can replay from any offset
Ordering	Global FIFO	Per-partition ordering
Use Case	Task/job processing	Event sourcing, analytics

8. Content Delivery Networks

How CDNs Work

Without CDN

User (Tokyo) → 12,000 km → Origin (NYC)

Latency: ~200ms

With CDN

User (Tokyo) → 50 km → Edge (Tokyo)

Latency: ~10ms

CDN EDGE NETWORK

```
┌──────────────────────────────────┐
│           CDN Network            │
│                                  │
│  ┌──────┐ ┌──────┐ ┌──────┐     │
│  │Tokyo │ │London│ │ NYC  │ 200+ │
│  │ Edge │ │ Edge │ │ Edge │ nodes │
│  └──┬───┘ └──┬───┘ └──┬───┘     │
│     └────────┼────────┘         │
│              │                   │
│          ┌───┴──────┐            │
│          │  Origin  │            │
│          │  Server  │            │
│          └──────────┘            │
└──────────────────────────────────┘
```

CDN Caching Flow

```
First Request (Cache MISS):
  User → Edge → Origin → Edge (caches response) → User

Subsequent Requests (Cache HIT):
  User → Edge → User   (origin never contacted!)
```

Major CDN Providers

Provider	Edge Locations	Notable Users
Cloudflare	300+ cities	~20% of all websites
AWS CloudFront	400+ PoPs	Amazon, Twitch, Hulu
Akamai	4,000+ locations	Apple, Microsoft, Adobe
Fastly	80+ PoPs	GitHub, Stripe, Spotify

9. Microservices Deep Dive

9.1 API Gateway Pattern

```
Mobile App  ──────►┌─────────────────┐──► User Service
Web App     ──────►│  API Gateway    │──► Product Service
Partner API ──────►│(Kong / Envoy)   │──► Order Service
IoT Devices ──────►└─────────────────┘──► Payment Service
```

Responsibilities: Authentication, Rate Limiting, Routing, Protocol Translation, Response Aggregation, SSL Termination, Logging

9.2 Service Discovery

```
┌──────────┐  1. Register  ┌──────────────┐
│ Service A│──────────────►│Service Registry│
│(3 inst.) │               │(Consul / etcd) │
└──────────┘               └──────┬───────┘
┌──────────┐  2. Lookup           │
│ Service B│──────────────►       │
│          │◄──────────────  A @ 10.0.1.5:8080
└──────────┘  3. Response    A @ 10.0.1.6:8080
```

9.3 Circuit Breaker Pattern

Prevents cascading failures when a downstream service is unhealthy.

CIRCUIT BREAKER STATE MACHINE

```
                  success
      ┌──────────────────────┐
      ▼        failure       │
  ┌─────────┐ (threshold) ┌──┴──────┐
  │ CLOSED  │────────────►│   OPEN  │
  │(normal) │             │(failing)│
  └─────────┘             └────┬────┘
                               │ timeout
                          ┌────▼──────┐
                          │ HALF-OPEN │
                          │ (testing) │
                          └───────────┘
                           success → CLOSED
                           failure → OPEN
```

9.4 Saga Pattern (Distributed Transactions)

CHOREOGRAPHY-BASED SAGA

```
Order Created → Payment Charged → Inventory Reserved → Shipping

If any step fails — compensating transactions roll back:
  Payment Failed?  → Cancel Order
  Stock Empty?     → Refund Payment
  Ship Failed?     → Release Reserve + Refund + Cancel
```

Netflix

The World's Leading Streaming Platform

280M+	190+	100M+	~15%
SUBSCRIBERS	COUNTRIES	HOURS/DAY STREAMED	GLOBAL INTERNET TRAFFIC

Architecture Overview

```
┌──────────────────────────────────────────────┐
│              NETFLIX ARCHITECTURE                 │
│                                                  │
│  ┌────────┐    ┌──────────┐    ┌──────────────┐ │
│  │ Client │──→│   AWS    │──→│ Open Connect │ │
│  │Devices │    │  Cloud   │    │  CDN (OCA)   │ │
│  │        │←──│          │←──│              │ │
│  └────────┘    └──────────┘    └──────────────┘ │
│                     │                            │
│                     │                            │
│                ┌────┴──────┐                     │
│                │Microservices│                    │
│                │   (1000+)  │                     │
│                └────────────┘                     │
└──────────────────────────────────────────────┘
```

1. Open Connect CDN

Netflix built its **own CDN** instead of relying on third-party providers.

OCA SERVERS SIT INSIDE ISP NETWORKS

```
Traditional:  User → ISP → Internet → CDN Edge → Origin
Netflix:      User → ISP → OCA Server (INSIDE ISP!) → User

┌──────────────────────────────┐
│        ISP Network           │
│  ┌────────────────────────┐  │
│  │  Open Connect Appliance  │  │
│  │  • 100-200 TB Storage    │  │
│  │  • Custom FreeBSD + nginx │  │
│  └────────────────────────┘  │
│  Users get content at LAN speed! │
└──────────────────────────────┘
```

- **18,000+** OCA servers in **6,000+** ISP locations worldwide
- Content pre-positioned during off-peak hours
- Handles **~95%** of all Netflix traffic
- Netflix provides hardware **free** to ISPs

2. Microservices on AWS

Netflix runs **1,000+ microservices** on AWS:

```
┌──────────────────────────────────────────────┐
│              API Gateway (Zuul)              │
│  Rate limiting · Authentication · Routing    │
└──────────────────────┬───────────────────────┘
        ┌──────────────┼──────────────┐
        ▼              ▼              ▼
  ┌──────────┐   ┌──────────┐   ┌──────────┐
  │  User    │   │ Content  │   │ Playback │
  │ Profile  │   │Discovery │   │ Service  │
  └────┬─────┘   └────┬─────┘   └────┬─────┘
       ▼              ▼              ▼
  ┌──────────┐   ┌──────────┐   ┌──────────┐
  │Cassandra │   │Elastic-  │   │ EVCache  │
  └──────────┘   │search    │   └──────────┘
                 └──────────┘
```

3. Recommendation Engine

```
User watches "Stranger Things"
       │
       ▼
┌──────────────────────────────────────┐
│  Collaborative Filtering             │
│  "Users who watched X also watched Y"  │
│            +                         │
│  Content-Based Filtering             │
│  "Sci-fi, 80s, supernatural"         │
│            │                         │
│            ▼                         │
│  ML Ranking Model (per-user)         │
└──────────────────────────────────────┘
       │
       ▼
  Personalized homepage with ranked rows
```

- **80%** of content watched comes from recommendations
- Even **artwork is personalized** — different users see different thumbnails
- Hundreds of A/B tests running simultaneously

4. Chaos Engineering

Tool	What It Does
Chaos Monkey	Randomly kills production instances
Chaos Gorilla	Takes down an entire availability zone
Chaos Kong	Takes down an entire AWS region
Latency Monkey	Introduces artificial network delays

> ***"The best way to avoid failure is to fail constantly."*** — *Netflix Engineering*

5. Netflix Tech Stack

Cloud	AWS	**CDN**	Open Connect (custom)
API Gateway	Zuul (open-sourced)	**Database**	Cassandra, MySQL, CockroachDB
Cache	EVCache (Memcached)	**Streaming**	Apache Kafka
Search	Elasticsearch	**CI/CD**	Spinnaker (open-sourced)

YouTube

The World's Largest Video Platform

2.7B+	1B+	500+	EB-scale
MONTHLY ACTIVE USERS	HOURS WATCHED / DAY	HOURS UPLOADED / MIN	VIDEO DATA STORED

Video Upload & Processing Pipeline

```
User uploads video → Upload Server (chunked)
        |
        ▼
┌──────────────────────────────────────────┐
|          VIDEO PROCESSING PIPELINE        |
|                                           |
|  Transcoding          Thumbnail Generation |
|  Original → 144p      AI picks best frames |
|          → 360p                           |
|          → 720p       Content ID (copyright) |
|          → 1080p      Match against database |
|          → 1440p                          |
|          → 2160p(4K)  Safety Check        |
|                       Nudity, spam, violence |
|  Audio (AAC, Opus)                        |
|  Auto-Captions (Speech-to-Text)           |
└──────────────────────────────────────────┘
        |
        ▼
Google Cloud Storage (each video in 20+ formats)
```

Adaptive Bitrate Streaming

```
User's bandwidth fluctuates during playback:

Time:     0s     10s     20s     30s     40s
Speed:   ████    ▓▓     ████   ██████   ▓▓
Quality: 720p   360p    720p   1080p   480p

1. Video split into small segments (2-10 seconds each)
2. Each segment encoded at multiple quality levels
3. Client monitors download speed in real-time
4. Client requests next segment at appropriate quality
5. Seamless quality switches — zero buffering
```

Vitess: YouTube's Database Solution

HORIZONTAL MYSQL SCALING

```
Problem: Single MySQL couldn't handle YouTube's scale
Solution: Vitess — database clustering system for MySQL

┌──────────────────────────────────────────┐
│                VITESS                    │
│  ┌──────────┐    ┌──────────────────┐    │
│  │  VTGate  │──→│  VTTablet Layer  │    │
│  │  (Proxy) │    │                  │    │
│  │ Looks    │    │  ┌──────┐ ┌──────┐  │    │
│  │ like a   │    │  │MySQL│ │MySQL│  │    │
│  │ single   │    │  │Shd 1│ │Shd 2│  │    │
│  │ MySQL    │    │  └──────┘ └──────┘  │    │
│  └──────────┘    └──────────────────┘    │
└──────────────────────────────────────────┘

Now open-source — used by Slack, GitHub, and others
```

Recommendation System

```
Watch History + Search History + Demographics
                  │
        ┌─────────▼──────────┐
        │ CANDIDATE GENERATION │  Millions → Hundreds
        │ (Deep Neural Network) │
        └────────────────────┘
                  │
        ┌─────────▼──────────┐
        │ RANKING              │  Hundreds → Ranked list
        │ (DNN + more features) │
        └────────────────────┘
                  │
        ┌─────────▼──────────┐
        │ DIVERSIFICATION      │  Apply business rules
        └────────────────────┘
                  │
          User's Homepage & "Up Next"
```

- **70%** of watch time comes from recommendations
- Re-ranks in **real-time** as user watches

YouTube Tech Stack

Cloud	Google Cloud (internal)	**Database**	Vitess, Bigtable, Spanner
Storage	GCS (Colossus)	**CDN**	Google Global Cache

Streaming	DASH / HLS adaptive	**ML/AI**	TensorFlow
Messaging	Google Pub/Sub	**Search**	Custom (Google infra)

Twitter / X

Real-Time Social Communication at Scale

500M+	500M+	300K+	20K+
MONTHLY ACTIVE USERS	TWEETS / DAY	TIMELINE REQ / SEC	PEAK TWEETS / SEC

Fan-out on Write (Push Model)

```
User A posts a tweet (A has 1,000 followers)
        │
   ┌────▼──────────┐
   │ Tweet Service  │── Store tweet in tweets table
   └───────┬───────┘
   ┌────▼──────────┐
   │ Fan-out       │── For EACH follower:
   │ Service       │   Insert tweet ID into their timeline cache
   └───────┬───────┘
        ├──► Follower 1's timeline cache: [tweet_new, ...]
        ├──► Follower 2's timeline cache: [tweet_new, ...]
        └──► ... (1,000 cache writes)

When follower opens app → read pre-computed cache → Instant!
```

The Celebrity Problem

HYBRID PUSH/PULL APPROACH

```
Problem: @Cristiano has 170M followers
  Fan-out on write = 170,000,000 cache writes per tweet!

Solution: HYBRID approach
  Regular users (fewer followers) → Fan-out on WRITE (push)
  Celebrities (millions followers) → Fan-out on READ (pull)

Your Timeline =
  Pre-computed cache (regular follows)
  + Real-time merge (celebrity tweets)
  + Ranking & filtering
```

Snowflake ID Generation

```
64-bit unique ID structure:
┌─────────┬────────────────┬────────────┬───────────────┐
│Sign (1)│ Timestamp (41)│Machine (10)│Sequence (12) │
└─────────┴────────────────┴────────────┴───────────────┘

• 4,096 unique IDs per millisecond per machine
• Time-sortable: newer tweets = larger IDs
• No coordination between machines needed
```

Twitter Tech Stack

Backend	Java (JVM), Scala	**Database**	Manhattan (custom)
Cache	Memcached, Redis	**Search**	Earlybird (Lucene)
Messaging	Kafka, EventBus	**ID Gen**	Snowflake (open-sourced)

Amazon

E-commerce Giant & Cloud Pioneer

350M+	66K+	$570B+	30+
PRODUCTS LISTED	ORDERS / SEC (PEAK)	ANNUAL REVENUE	AWS REGIONS

Architecture Evolution

```
2001: Monolith                    2024: Microservices
┌─────────────────┐               ┌─┐ ┌─┐ ┌─┐ ┌─┐
│   ONE GIANT     │    ──►        ├─┤ ├─┤ ├─┤ ├─┤
│   APPLICATION   │               ├─┤ ├─┤ ├─┤ ├─┤
└─────────────────┘               └─┘ └─┘ └─┘ └─┘
                                  1000s of services
```

Jeff Bezos' 2002 Mandate: *All teams must expose data through service interfaces. Teams communicate ONLY through interfaces. All interfaces must be externalizable. "Anyone who doesn't do this will be fired." — This mandate eventually became AWS.*

A Single Product Page = 100+ Services

```
┌──────────────────────────────────────────┐
│  amazon.com/product/B09V3KXJPB           │
│                                          │
│  Product Image Svc     Price Service     │
│  (images, 360° view)   (dynamic pricing) │
│                                          │
│  Review Service        Inventory Service │
│  (ratings, Q&A)        (stock status)    │
│                                          │
│  Shipping Service      Recommendation Svc│
│  (delivery dates)      ("Also bought...")│
│                                          │
│  Ads Service           Seller Service    │
│  (sponsored items)     (marketplace info)│
│                                          │
│  + 90 more services...                   │
└──────────────────────────────────────────┘
```

DynamoDB: Built for Scale

Built after the 2004 holiday season crash. Design principles:

- **Always writable** (availability over consistency)
- **Single-digit ms** latency guaranteed
- **Infinitely scalable** via horizontal partitioning
- **No single point of failure**

Amazon Tech Stack

Cloud	AWS (they built it!)	**Database**	DynamoDB, Aurora, RDS
Cache	ElastiCache	**Search**	OpenSearch
Messaging	SQS, SNS, Kinesis	**CDN**	CloudFront

Uber

Real-Time Ride Matching at Global Scale

25M+	70+	10K+	5M+
TRIPS / DAY	COUNTRIES	CITIES	ACTIVE DRIVERS

Core: Matching Riders to Drivers

```
Rider requests ride
       |
  ┌────▼──────────────────────────────────┐
  |      DISPATCH (Matching Engine)        |
  |                                        |
  |  1. Query nearby drivers (geospatial) |
  |  2. Calculate ETA for each driver     |
  |  3. Rank by: ETA, rating, vehicle     |
  |  4. Send request to best match        |
  |  5. If declined → try next best       |
  |                                        |
  |  Latency budget: < 200ms              |
  └────────────────────────────────────────┘
       |
  Driver accepts → Trip starts
```

Geospatial Indexing with H3

Uber uses **H3** (Hexagonal Hierarchical Spatial Index):

```
┌──────────────────────────────────────┐
|  City Map with H3 Hexagonal Grid      |
|                                       |
|    /\    /\    /\    /\              |
|   /  \  /  \  /  \  /  \             |
|  / D1 \/ D2 \/    \/    \            |
|  \    /\    /\    /\ D3 /            |
|   \  / R\  /  \  /  \  /             |
|    \/    \/    \/    \/              |
|                                       |
|  R = Rider    D = Available drivers   |
|  Search: Find hex → query cell → expand |
|  O(1) lookup per cell!                |
└──────────────────────────────────────┘
```

Surge Pricing

```
Demand ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  (100 requests)
Supply ▓▓▓▓              (25 drivers)
Ratio = 4.0 → Surge multiplier ~2.5x

Updates every ~2 minutes per geographic zone
Inputs: demand, supply, weather, events, traffic
```

Real-Time Data Platform

```
Driver GPS (every 4s) ──►┌──────────┐──► Real-time Analytics (Flink)
Rider Events ──────────►│ Kafka    │──► Batch Analytics (Spark/Hive)
Trip Events ───────────►│          │──► ML Pipeline (pricing, ETA)
                        └──────────┘   Trillions of events
```

WhatsApp

Simplicity at Unprecedented Scale

2.7B+	100B+	~50	~550
MONTHLY ACTIVE USERS	MESSAGES / DAY	ENGINEERS (AT ACQ.)	SERVERS (AT ACQ.)

> *WhatsApp achieved one of the most impressive **engineer-to-user ratios** in tech history: ~50 engineers serving 2+ billion users.*

Architecture Philosophy: SIMPLICITY

What They Used

- Erlang/OTP for concurrency
- FreeBSD (tuned networking)
- Mnesia (distributed Erlang DB)
- Custom XMPP protocol
- Single-digit ms delivery

What They Avoided

- No microservices
- No complex infrastructure
- No heavy frameworks
- No unnecessary abstractions

Message Flow

```
Alice sends "Hello!" to Bob
        |
  Alice's Phone (encrypts message end-to-end)
        |
  ┌─────▼──────────────────────────────────────┐
  |          WhatsApp Server (Erlang)           |
  |                                             |
  |  1. Receive encrypted message               |
  |  2. Look up Bob's connection server         |
  |  3. Route to Bob's server                   |
  |                                             |
  |  Bob ONLINE:  Deliver instantly → Delete    |
  |  Bob OFFLINE: Store temp → Deliver later    |
  |                                             |
  |  Server NEVER sees plaintext content!       |
  └─────────────────────────────────────────────┘
        |
  Bob's Phone (decrypts message)
```

Why Erlang?

Traditional (Thread-based)

Each connection = 1 OS thread

1 server → **~10,000** connections

Erlang (Actor-based)

Each connection = 1 process (< 1KB)

1 server → **2,000,000+** connections!

Erlang processes are fully isolated — if one crashes, all others are unaffected. This makes the system inherently fault-tolerant.

16. System Design Best Practices

16.1 The Twelve-Factor App

#	Factor	Principle
I	**Codebase**	One codebase, many deploys
II	**Dependencies**	Explicitly declare & isolate
III	**Config**	Store config in the environment
IV	**Backing Svcs**	Treat as attached resources
V	**Build/Release**	Strictly separate stages
VI	**Processes**	Execute as stateless processes
VII	**Port Binding**	Export services via port binding
VIII	**Concurrency**	Scale out via process model
IX	**Disposability**	Fast startup, graceful shutdown
X	**Dev/Prod Parity**	Keep environments similar
XI	**Logs**	Treat as event streams
XII	**Admin**	Run admin tasks as one-off processes

16.2 Design for Failure

> ***"Everything fails, all the time."*** — *Werner Vogels, Amazon CTO*

Failure Scenario	Defense Strategy
Server crash	Redundancy + auto-restart
Network partition	Retry with backoff + circuit breaker
Database failure	Replicas + automatic failover
Data center outage	Multi-AZ deployment
Region outage	Multi-region deployment
Bad deployment	Blue-green + instant rollback
Traffic spike	Auto-scaling + rate limiting

16.3 Rate Limiting

Token Bucket

```
Bucket capacity: 10
●●●●●●●○○○  (7 tokens)
Fill rate: 1 token/second
Request → consume 1 token
Empty → reject (HTTP 429)
```

Sliding Window

```
Window: last 60 seconds
14 req / limit 20 → ALLOWED
21 req / limit 20 → BLOCKED
Smooths out bursty traffic
```

16.4 Consistency Patterns

Pattern	Behavior	Example
Strong	Write instantly visible to ALL	Bank balance
Eventual	Visible to all EVENTUALLY	Social media likes
Read-Your-Writes	Writer sees own update immediately	Profile edit
Causal	If B depends on A, see A before B	Reply after comment

17. Monitoring, Observability & Reliability

The Three Pillars of Observability

Pillar	What It Measures	Example Tools
Metrics	CPU, memory, latency, error rate, request rate	Prometheus, Datadog, Grafana
Logs	Error messages, stack traces, audit trails	ELK Stack, Splunk, CloudWatch
Traces	Request flow across multiple services	Jaeger, Zipkin, Datadog APM

SLIs, SLOs, and SLAs

Term	Definition	Example
SLI	Service Level Indicator (measurable metric)	"99.2% of requests < 200ms"
SLO	Service Level Objective (internal target)	"We aim for 99.9% < 200ms"
SLA	Service Level Agreement (external contract)	"99.95% uptime guaranteed"

Availability Nines

Level	Uptime %	Downtime Per Year
Two nines	99%	3.65 days
Three nines	99.9%	8.76 hours
Four nines	99.99%	52.6 minutes
Five nines	99.999%	5.26 minutes

Deployment Strategies

```
Blue-Green Deployment:
  Blue (v1) ◄── 100% traffic ── Load Balancer
  Green (v2) ← deploy here, test, then switch all traffic

Canary Deployment:
  Stable (v1) ◄── 95% traffic ── Load Balancer
  Canary (v2) ◄──  5% traffic    (monitor, gradually increase)

Rolling Update:
  Instance 1: v1 → v2  (updating)
  Instance 2: v1       (still serving)
  Instance 3: v1       (still serving)
  ... update one-by-one until all on v2
```

18. Summary & Key Takeaways

Architecture Decision Matrix

Scale Level	Recommended Architecture
0 – 1K users	Single server, monolith, single database
1K – 100K	Load balancer, read replicas, CDN
100K – 1M	Microservices, caching layer, message queues
1M – 10M	Database sharding, multi-region, event streaming
10M – 100M	Custom infrastructure, edge computing
100M+	Build your own (Netflix, Google, Meta approach)

Key Lessons from Each Company

Company	Key Takeaway
Netflix	Build your own CDN when content delivery is core. Chaos engineering — test failure actively. Personalization drives 80% of views.
YouTube	Design for massive write throughput (500 hr/min). Adaptive streaming for variable networks. Custom DB when needed (Vitess).
Twitter	Fan-out strategies matter for social graphs. Hybrid push/pull beats pure approaches. Snowflake IDs for distributed systems.
Amazon	Microservices mandate changed the industry. Build internal tools, then sell them (AWS). Every page = 100+ service calls.
Uber	Geospatial indexing (H3) for location apps. Real-time data at massive scale. Dynamic pricing requires real-time ML.
WhatsApp	Simplicity wins — 50 engineers, 2B+ users. Right language for the job (Erlang). E2E encryption as architecture.

The 10 Golden Rules of System Design

1. **Keep It Simple**
   Start with a monolith, evolve to microservices as needed

2 **Design for Failure**

Everything fails. Plan for it. Test for it. Automate recovery.

3 **Scale Horizontally**

Vertical scaling hits limits. Horizontal scaling is nearly infinite.

4 **Cache Aggressively**

The fastest database query is the one you never make.

5 **Use Async Where Possible**

Message queues decouple services and buffer traffic spikes.

6 **Monitor Everything**

You can't fix what you can't see. Metrics, logs, traces.

7 **Automate Deployment**

Blue-green, canary, rolling. Never deploy manually.

8 **Choose the Right Database**

SQL for transactions, NoSQL for scale, Graph for relationships.

9 **Secure by Default**

Encryption at rest + in transit. Zero trust. Least privilege.

10 **Measure, Then Optimize**

Profile first. Don't guess where the bottleneck is.

System Architecture: A Comprehensive Guide

April 2026 Edition